Exp: 12
24/9/26

a) Implementation of Echo Client Server

Aim:

To implement echo client server using TCP/UDP sockets.

Algorithm:

i) TCP:

Echo Server and ~~Server~~ Client:

* Start the TCP echo server and set the server port to the given port or use 5000 by default.
* Create a ServerSocket and listen on the specified port.
* Wait for the client connection and create input and output streams for communication with client.
* Read a message sent by the client and display the received message and send it back to the client with "Echo: " added.
* If the received message is bye, close connection. Otherwise continue receiving messages.

ii) UDP:

Echo S

* Start
86000.
* Start
packet.
* Read
into a
* The se
message
and send
* The c
client

ii) UDP :

Echo Server and Client

* Start the UDP server and listen on port 86000.
* Start the UDP client and create a datagram packet.
* Read the message from the client and convert it into a datagram packet and send it to the server.
* The server receives the packet and extracts the message. The server then adds "Echo:" to the packet and sends back the response.
* The client receives the echoed response. If the client sends bye, stop communication.

Code: TCP

TCP Echo Client

```java
import java.io.*;
import java.net.*;

public class TCPEchoClient {
    Run | Debug
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;

        try (Socket socket = new Socket(host, port);
             PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {

            System.out.println("Connected to TCP Echo Server at " + host + ":" + port);
            System.out.println(x: "Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                out.println(userInput);
                String response = in.readLine();
                System.out.println("Server: " + response);
                if (userInput.equalsIgnoreCase(anotherString: "bye")) {
                    break;
                }
            }
        }
        System.out.println(x: "Connection closed.");
    }
}
```

TCP Echo Server

```java
import java.io.*;
import java.net.*;

public class TCPEchoServer {
    Run | Debug
    public static void main(String[] args) throws IOException {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo Server listening on port " + port);

            while (true) {
                try (Socket clientSocket = serverSocket.accept()) {
                    System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());

                    BufferedReader in = new BufferedReader(
                            new InputStreamReader(clientSocket.getInputStream()));
                    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), autoFlush: true);

                    String line;
                    while ((line = in.readLine()) != null) {
                        System.out.println("Received: " + line);
                        out.println("Echo: " + line);
                        if (line.equalsIgnoreCase(anotherString: "bye")) {
                            break;
                        }
                    }
                    System.out.println("Client disconnected: " + clientSocket.getRemoteSocketAddress());
                } catch (IOException e) {
                    System.out.println("Connection error: " + e.getMessage());
                }
            }
        }
    }
}
```

Output :

ECHO CLIENT

```
devyesh@Macs-MacBook-Air Downloads % javac TCPEchoClient.java
devyesh@Macs-MacBook-Air Downloads % java TCPEchoClient
Connected to TCP Echo Server at localhost:5000
Type messages to send (type 'bye' to quit):
Hello!
Server: Echo: Hello!
This is Echo Client, Do you copy ?
Server: Echo: This is Echo Client, Do you copy ?
Hello!
Server: Echo: Hello!
```

ECHO SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac TCPEchoServer.java
devyesh@Macs-MacBook-Air Downloads % java TCPEchoServer
TCP Echo Server listening on port 5000
Client connected: /127.0.0.1:49419
Received: Hello!
Received: This is Echo Client, Do you copy ?
Received: Hello!
```



Code : UDP

UDP Echo Client

```
import java.io.*;
import java.net.*;

public class UDPEchoClient {
    Run | Debug
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 6000;

        InetAddress address = InetAddress.getByName(host);

        try (DatagramSocket socket = new DatagramSocket();
             BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {

            socket.setSoTimeout(timeout: 5000); // avoid blocking forever if a reply is lost
            System.out.println("UDP Echo Client ready to send to " + host + ":" + port);
            System.out.println(x: "Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                byte[] sendData = userInput.getBytes();
                DatagramPacket request = new DatagramPacket(sendData, sendData.length, address, port);
                socket.send(request);

                byte[] buffer = new byte[1024];
                DatagramPacket response = new DatagramPacket(buffer, buffer.length);
                try {
                    socket.receive(response);
                    String reply = new String(response.getData(), offset: 0, response.getLength());
                    System.out.println("Server: " + reply);
                } catch (SocketTimeoutException e) {
                    System.out.println(x: "No response received (timeout).");
                }

                if (userInput.equalsIgnoreCase(anotherString: "bye")) {
                    break;
                }
            }
        }
        System.out.println(x: "Client finished.");
    }
}
```

UDP Echo Server

```
import java.net.*;

public class UDPEchoServer {
    Run | Debug
    public static void main(String[] args) throws Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
        byte[] buffer = new byte[1024];

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Echo Server listening on port " + port);

            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                String received = new String(request.getData(), offset: 0, request.getLength());
                InetAddress clientAddress = request.getAddress();
                int clientPort = request.getPort();
                System.out.println("Received from " + clientAddress + ":" + clientPort + " -> " + received);

                byte[] responseData = ("Echo: " + received).getBytes();
                DatagramPacket response = new DatagramPacket(
                        responseData, responseData.length, clientAddress, clientPort);
                socket.send(response);
            }
        }
    }
}
```

Output :

ECHO CLIENT

```
devyesh@Macs-MacBook-Air Downloads % javac UDPEchoClient.java
devyesh@Macs-MacBook-Air Downloads % java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello!
Server: Echo: Hello!
Request permission to authorize
Server: Echo: Request permission to authorize
Permission granted
Server: Echo: Permission granted
```

ECHO SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac UDPEchoServer.java
devyesh@Macs-MacBook-Air Downloads % java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:50050 -> Hello!
Received from /127.0.0.1:50050 -> Request permission to authorize
Received from /127.0.0.1:50050 -> Permission granted
Received from /127.0.0.1:50050 -> quit
Received from /127.0.0.1:50050 -> bye
```



Result :

The Implementation of Echo Client Server using TCP and UDP has been completed successfully.

Expt: 12
24/9/26

b) Implementation of Chat Program

Aim:

To implement a Chat Program using socket programming that enables communication between a client and a server through a network connection.

Algorithm:

i) Start the chat server on port 5000 and wait for client connection.

ii) Ask the client to enter a name and add the client to the connected-client list.

iii) Start the client and establish a socket connection with the server.

iv) The client sends messages entered by the user to the server.

v) The server receives each message and broadcasts it to all other connected clients.

vi) The client continuously listens for messages from the server using a seperate listener thread.

vii) If a client sends "/quit", terminate that clients connection.

viii) Remove the disconnected client and notify the remaining clients.

ix) Continue accepting and managing new clients until the server is stopped.

Code:

Chat Client

```
import java.io.*;
import java.net.*;

public class ChatClient {

    private static final String SERVER_ADDRESS = "127.0.0.1";
    private static final int SERVER_PORT = 5000;

    Run | Debug
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
        BufferedReader console = new BufferedReader(new InputStreamReader(System.in));

        Thread listener = new Thread(() -> {
            String serverMessage;
            try {
                while ((serverMessage = in.readLine()) != null) {
                    System.out.println(serverMessage);
                }
            } catch (IOException e) {
                System.out.println(x: "Disconnected from server.");
            }
        });
        listener.setDaemon(on: true);
        listener.start();

        String userInput;
        while ((userInput = console.readLine()) != null) {
            out.println(userInput);
            if (userInput.equalsIgnoreCase(anotherString: "/quit")) {
                break;
            }
        }

        socket.close();
    }
}
```

Chat Server

```java
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

public class ChatServer {
   private static final int PORT = 5000;
   private static final Set<ClientHandler> clients = Collections.synchronizedSet(new HashSet());

   public ChatServer() {
   }

   public static void main(String[] var0) throws IOException {
      ServerSocket var1 = new ServerSocket(5000);
      System.out.println("Chat server started on port 5000");

      while(true) {
         Socket var2 = var1.accept();
         ClientHandler var3 = new ClientHandler(var2);
         clients.add(var3);
         (new Thread(var3)).start();
      }
   }

   private static void broadcast(String var0, ClientHandler var1) {
      synchronized(clients) {
         for(ClientHandler var4 : clients) {
            if (var4 != var1) {
               var4.send(var0);
            }
         }

      }
   }

   private static void removeClient(ClientHandler var0) {
      clients.remove(var0);
      broadcast(var0.getName() + " has left the chat.", var0);
      System.out.println(var0.getName() + " disconnected.");
   }

   private static class ClientHandler implements Runnable {
      private final Socket socket;
      private BufferedReader in;
      private PrintWriter out;
      private String name;
```

```
        ClientHandler(Socket socket) {
            this.socket = socket;
        }

        String getName() {
            return name;
        }

        void send(String message) {
            out.println(message);
        }

        @Override
        public void run() {
            try {
                in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                out = new PrintWriter(socket.getOutputStream(), autoFlush: true);

                out.println(x: "Enter your name:");
                name = in.readLine();
                if (name == null) {
                    name = "Anonymous-" + socket.getPort();
                }

                System.out.println(name + " connected from " + socket.getInetAddress());
                broadcast(name + " has joined the chat.", this);
                out.println("Welcome, " + name + "! Type /quit to exit.");

                String line;
                while ((line = in.readLine()) != null) {
                    if (line.equalsIgnoreCase(anotherString: "/quit")) {
                        break;
                    }
                    System.out.println(name + ": " + line);
                    broadcast(name + ": " + line, this);
                }
            } catch (IOException e) {
                System.out.println("Connection error with " + name + ": " + e.getMessage());
            } finally {
                closeConnection();
            }
        }

        private void closeConnection() {
            try {
                removeClient(this);
                socket.close();
            } catch (IOException e) {
                // ignore
            }
        }
    }
}
```

Exp: 13
1/10/26

Output :

CHAT CLIENT

```
devyesh@Macs-MacBook-Air Downloads % java ChatClient
Enter your name:
Devyesh
Welcome, Devyesh! Type /quit to exit.
Hello
I am Devyesh
Nice to meet you
Bye!
/quit
Disconnected from server.
```

CHAT SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac ChatServer.java
devyesh@Macs-MacBook-Air Downloads % java ChatServer
Chat server started on port 5000
Devyesh connected from /127.0.0.1
Devyesh: Hello
Devyesh: I am Devyesh
Devyesh: Nice to meet you
Devyesh: Bye!
Devyesh disconnected.
```

Result :

The Implementation of Chat-Program has been completed successfully.